Complete Project Explanation

Rapid, Proximity-Bound, Human-Assisted Aggregation of Identities over Unreliable Short-Range Wireless Channels

PART 1 — What Is This Project And Why Does It Exist?

Every existing attendance system is fundamentally flawed:

- **Manual Roll Call:** Wastes **10–15 minutes** of every lecture. In a standard 50-minute session, that accounts for **20–30% of total teaching time lost**. Beyond the inefficiency, it is trivially defeated by "proxying," where a student simply calls out "present" for an absent peer.
- **Face Recognition Systems:** While seemingly modern, these introduce significant friction. Passing a scanner device around **disturbs the flow of the class**, while fixed-point scanners create **massive crowds** at the entrance. Furthermore, the **initial capital expenditure** for specialized cameras and the ongoing **maintenance costs** make it a heavy financial burden for institutions.
- **RFID Cards:** These verify the *card*, not the *student*. Cards are frequently handed to friends, as the system has no way to bind the physical token to the actual registered individual.
- **Stationary Biometric Scanners:** Fingerprint or iris scanners at the door create **unavoidable bottlenecks**. They fail to scale for large classrooms (100+ students) and are prohibitively expensive to install and network in every single classroom on a large campus.
- **Basic BLE Beacon Systems:** This modern approach has been conclusively defeated in research. Because basic BLE signals are passive, a student sitting in the hallway or even **forwarding the signal over the internet** via a friend's phone can be marked present without ever entering the room.

What it is: A high-security, decentralized attendance system that replaces manual roll calls and static hardware with a **Self-Organizing Connection Graph**. Instead of a central scanner, the system validates presence through a distributed network of **Peer-to-Peer (P2P) Handshakes**. As students connect, they form a live authentication mesh that covers the entire classroom automatically.

The Intuitive Flow

Imagine the classroom environment once the session begins:

- **The Targeted Signal:** A student who is already "validated" displays a dynamic QR code. This isn't just a code; it's a **Targeted Signal Beacon**. When a neighbor scans it, their phone ignores the "noise" of 50 nearby devices and locks onto that specific peer instantly.
- **The Graph Extension:** The scan initiates an automatic background handshake. Instead of a manual "pairing" process, the QR code provides the precise cryptographic handshake credentials needed to **attach** the new student to the existing secure graph.

- **The Secure Link:** Boom—the connection is established. The new student is now a verified node in the classroom network, capable of extending the signal to the next person. No menus, no ID entry, and no infrastructure lag.

Building the Class Mesh

The app automatically builds a "Connection Graph" based on who is sitting near whom:

- **The Chain:** You connect to the student on your **left** and the student on your **right**.
- **The Grid:** If there is no one to your right (end of the row), you connect to the student in **front** or **behind** you.
- **The Result:** Within seconds, the entire room is linked together like a giant net. No student is left as an isolated "island."

The Master Node Validation (BST-Driven)

The **Master Node** (the Professor's phone) is the only authorized "Root" for the attendance session.

- **The Root of Trust:** The Professor "signs" the initial layer of students in their immediate proximity.
- **The Backend Chain:** Once a student connects to the Master Node, the **Backend** processes the connection using a **Binary Search Tree (BST)** or Graph-based logic.
- **Instant Validation:** Whoever is connected to the Master Node—either directly or through a chain of peers—is automatically identified by the BST as a valid node and marked **Present**.

  Simply Put: If the system can trace a path from your phone back to the Master Node through the backend's tree, you are officially in the room. No connection to the Master Node means no attendance.

The Biometric "Gate"

But before that QR even shows up, the app asks for a **fingerprint**.

- **The Security:** This isn't just a simple login. The phone's hardware checks if the fingerprint on the sensor matches, android give a 'yes' signal
- **The "Anti-Cheat":** If a student tries to be "smart" by adding their friend's fingerprint to their phone settings, the **Android Keystore** (the phone's internal vault) detects the change. It will essentially "lock the vault," making it impossible to sign the attendance token because the biometric profile has been tampered with.

The Device Anchor (Hardware Locking)

To ensure the integrity of the connection graph, the system enforces a strict **1:1 Device-to-Student Mapping** at the pre-login stage.

- **Identity Anchoring:** Upon first use, a student's account is cryptographically "anchored" to their phone's unique hardware signature (UUID/IMEI).
- **Pre-Login Verification:** Before allowing a session to start, the app cross-references the current device against this stored anchor. If the hardware IDs do not match, the login is blocked.

Google Play Integrity API (Proves The App Is Genuine And Unmodified)

What it protects against: An attacker downloading the app, reverse-engineering it, removing the proximity checks, and reinstalling the modified version.

This is called a "modified client attack" and is one of the most serious threats to any mobile security system. A modified app could simply skip all checks and directly send "I am present" to the server.

How it works:

Before the server sends any session keys to any student, it first asks Google to verify the app. Google's Play Integrity API checks three things about the app and device:

- **Question 1: Was this app downloaded from the official Google Play Store?** *(Ensures the app was not sideloaded from a suspicious APK file.)*
- **Question 2: Is the app's code unmodified and identical to the published version?** *(Ensures there is no tampering, no removed security checks, and no injected malicious code.)*
- **Question 3: Is the device a real, non-rooted Android phone?** *(Ensures the app is not running on an emulator, a virtual environment, or a device with root access that could bypass hardware security.)*

Google returns a digitally signed certificate answering these questions. The Django backend verifies this certificate with Google before distributing any keys.

Security Analysis: The Impossibility of Integrity Forgery

1. Hardware-Backed Kill Switches

Modern smartphones are designed with **Write-Once-Read-Many (WORM)** components, such as e-fuses.

- **The Mechanic:** Attempting to unlock the bootloader (a prerequisite for rooting) physically trips a hardware "kill switch" (e.g., Samsung Knox bit).
- **The Impact:** This change is physical and irreversible. Once tripped, the hardware's **Trusted Execution Environment (TEE)** will forever report the device as "Tampered." No software patch or "root hide" tool can reset a physical fuse.

2. Cryptographic Attestation (The Impossible Forge)

The API does not rely on the Android OS to check for root (which could be lied to). Instead:

- The device's **Secure Element (SE)** signs a unique challenge using a private key fused into the silicon during manufacturing.
- **The Barrier:** An attacker would need to extract a private key from a protected hardware chip—a feat that requires multi-million dollar lab equipment and has no known software-based workaround. Without this key, a "fake" integrity signal cannot be signed.

3. Remote Verification (Zero-Trust Architecture)

The validation of the device "verdict" happens **off-device**.

- Even if a student successfully modifies their local system files to *pretend* the phone is secure, the **Google Backend** and our **Django Server** perform a server-to-server handshake.
- Because the verification happens in the cloud, the student's local "hacks" never touch the decision-making logic.

4. The "Security vs. Utility" Trade-off

For a student, the "cost" of attempting to bypass this system is catastrophic:

- **Permanent Devaluation:** Bypassing security often permanently disables banking apps (UPI, GPay), streaming (Netflix HD), and official OS updates.
- **Risk:** The technical complexity required to even *attempt* a bypass on modern Android versions (Android 13+) usually results in a **"Hard Brick,"** rendering the hardware useless.

Conclusion: By enforcing **Strong Integrity**, the system moves security from the "malleable" software layer to the "immutable" hardware layer. This renders unauthorized environment modification (rooting) or API spoofing **computationally and physically infeasible** for the end-user.

RSSI Proximity Check

The system reads the RSSI (Received Signal Strength Indicator) value of the BLE connection immediately after the handshake completes. RSSI measures how strong the incoming signal is in dBm — the less negative the value, the closer the device.

```
## How RSSI Works As A Distance Check
```
Distance   Typical RSSI   Meaning
____
0 – 1 metre   -40 to -55 dBm   Same desk, very close
1 – 3 metres   -55 to -70 dBm   Normal classroom neighbour
3 – 5 metres   -70 to -80 dBm   Getting far, borderline
Outside room   -80 to -95 dBm   Through wall, too far
No signal   below -95 dBm   Definitely not present
```
```

RSSI catches the simpler case of a student sitting just outside the room or corridor trying to reach the mesh without being physically present

Session-Scoped Ephemeral HOTP (Prevents Replay Attacks)

What it protects against: Recording a valid attendance token today and replaying it the session in next try .

The key distribution:

Before each lecture, the Django server generates a **unique session seed** for every individual student. "Unique" means two things here — different for every student, and different for every lecture.

Formula used by server:

```
session_seed[student] = HMAC( master_session_key, student_UID )
```

This gives:

```
Piyush,  Monday 9AM:   seed = 0xA3F7B2C1...

Piyush,  Monday 11AM:  seed = 0xD841F903...  ← completely different
```

Each seed is encrypted with the student's public key and sent over HTTPS before the session. The student's app stores it in secure storage. The server then goes quiet — it has no further involvement until the final confirmation step.

The HOTP counter — why a simple number is dangerous:

If the system sent a simple counter like 1, 2, 3... an attacker observing counter value 5 immediately knows the next value is 6. This allows forging tokens.

Instead, the app never transmits the counter n directly. It transmits a derived value called an HOTP token:

token = HMAC-SHA256(session_seed, n)

replay across sessions is not possible:

LAYER 4 — BLE GATT Handshake With RTT Bounding (The Core Anti-Relay Mechanism)

What GATT is:

GATT stands for Generic Attribute Profile. It is a structured BLE communication protocol where:

- One device runs a **GATT Server** — it exposes data "characteristics" (think of these as named variables that can be read or written)
- Another device runs a **GATT Client** — it connects to the server and reads/writes those characteristics

The complete 3-step handshake:

This is happening between two student's phone-

```
STUDENT APP (GATT Client)                FACULTY APP (GATT Server)
─────────────────────────                ─────────────────────────

STEP 1 — Student sends encrypted challenge

  Compute:
    token  =  HMAC-SHA256( session_seed, n )

  Build payload (92 bytes total):
    payload = AES-128-GCM-Encrypt( K1,
                token         ← 32 bytes
                student_UUID  ← 16 bytes
              )
    (+12 IV + 16 GCM auth tag = 92 bytes total)

  → Write payload to GATT Characteristic
  → START RTT TIMER ⏱


STEP 2 — Server echoes back with identity stamp

                                Server decrypts with K1
                                Gets: token, student_UUID

                                CRITICAL: Server does NOT verify token.
                                Server has no session_seed.
                                Server cannot check if HMAC is correct.
                                Server only stamps its own identity:

                                response = AES-128-GCM-Encrypt( K1,
                                  server_UUID    ← 16 bytes, who the server is
                                  token          ← 32 bytes, echoed unchanged
                                  session_id     ← 16 bytes, current lecture
                                )
                                (+12 IV + 16 GCM auth tag = 108 bytes total)

                                → Write response to GATT Characteristic
```

```
STEP 3 — Student performs all three checks

  → STOP RTT TIMER ⏱

  CHECK 1 — RTT < 150ms ?
    Total transmission time for 200 bytes over BLE is ~2ms.
    That leaves 148ms for processing.
    An internet relay adds a minimum of 50-200ms just for the round trip.
    Any relay attack pushes RTT over 150ms → RELAY DETECTED → ABORT
    ✅ Pass: device is physically close
    ❌ Fail: relay attack → abort

  Decrypt response with K1:
  (If any byte was tampered with, AES-GCM auth tag fails here
   and decryption is rejected before the checks even run)

  CHECK 2 — server_UUID matches pre-registered faculty UUID ?
    Faculty UUID was stored during app registration.
    A fake GATT server (ESP32, another phone) will have a different UUID.
    ✅ Pass: genuine server confirmed
    ❌ Fail: spoofing attack → abort

  CHECK 3 — token_echo matches HMAC(session_seed, n) that I computed?
    This check is performed entirely on the client side.
    The server echoed the token without being able to verify it.
    Only the genuine student app holding session_seed can confirm this.
    Since n increments after every successful handshake, a replayed
    old response will always have a stale token that does not match
    the current n — making replay attacks impossible without the nonce.
    ✅ Pass: packet integrity + freshness confirmed
    ❌ Fail: tampering or replay detected → abort

  ALL 3 CHECKS PASS:
    n = n + 1  ← increment counter
    Handshake marked as VERIFIED
    Send confirmation to Django via HTTPS
```

Why The Fallback Mode Is A Practical Necessity

The full GATT handshake mode requires a device to reliably run a BLE GATT server, handle concurrent client connections, and maintain stable advertising intervals under load. During testing, GATT server functionality was found to be unstable on at least one device, failing to advertise correctly or dropping connections unexpectedly. Whether this is limited to specific older or budget models, a firmware issue, or something that resolves on newer hardware is not yet conclusively known — the current dataset from testing is too small to make a definitive claim.

Simplified Fallback Mode — BLE Mesh + QR + Encrypted Token + GPS

The simplified fallback mode keeps the full BLE mesh topology intact — students still connect to their left, right, front, and back neighbours exactly as in the primary mode, and the BFS ripple propagation on the backend remains unchanged. The only difference is in how the student-to-student token exchange works. Instead of a full GATT server handshake with RTT bounding, each student scans their neighbour's QR code to get their UUID and session token,

encrypts the combined payload with the shared symmetric key K1, and sends it directly to the Django server over HTTPS after decrypting. The server verifies both UUIDs and draws the edge in the proximity graph as normal.

There is no GATT server setup, no three-step handshake, and no RTT measurement only,rssi. This makes implementation simpler and works on a wider range of devices. The proximity guarantee is softer — it relies on the student physically scanning a neighbour's QR code at close range, supported by GPS radius verification and developer mode detection. Since the QR changes every session and the payload is encrypted with K1, a student who was not physically present cannot forge a valid token. The main weakness compared to the full mode is the absence of RTT bounding, meaning a sufficiently motivated attacker could attempt a relay — which is why this remains the fallback rather than the default.

Property	Full GATT Mode	Simplified Fallback Mode
BLE mesh topology	✅ Present	✅ Present
BFS ripple propagation	✅ Present	✅ Present
Student-to-student exchange	GATT 3-step handshake	QR scan + encrypted token
Relay attack resistance	RTT bounding (hard block)	GPS radius check (soft check)
GATT server required	Yes	No
Security level	High	Moderate

Exception Handling & Fail-Safe Mechanisms

No system is perfect, and academic environments require flexibility for hardware issues. The following protocols handle anomalies without compromising security.

1. The "Delegate Master Node" Protocol

If the Faculty's device experiences hardware failure (BLE chip error, battery depletion, or GATT server crash), the professor can use their Web Dashboard or another device to promote a specific student to a **Temporary Master Node**.

- **The Delegation:** The Django backend marks `Student_X` as the **Root** for `Session_ID`.
- **The Ripple Change:** The BFS Ripple Engine on the backend updates its starting coordinate. Instead of searching from the Faculty UUID, it initiates the BFS from `Student_X_UUID`.
- **Trust Anchor:** Because `Student_X` was already verified via the Play Integrity API and their own Biometric check, they are a trusted "Proxy Root."
- **Verification:** Attendance is marked for everyone reachable from this new student "Master."

Human-Assisted Exception Protocol (The "Manual Bypass")

This layer handles the "edge cases"—students who are physically present but cannot complete the digital handshake due to technical issues (dead battery, broken BLE, or lost phone).

1. The "Flag for Review" Workflow

Instead of a student just shouting "present," the process is digitized and peer-verified to save time:

- **The Claim:** The student with the issue informs a nearby friend.
- **Peer Exception Log:** The friend opens their app and enters the UID of the student with the issue into an **Exception Request** field.
- **The Proximity Proof:** Because the friend is already "Verified" via the BLE handshake, the system knows that anyone they flag is *at least* physically near a verified device.

2. The Faculty "Fast-Roll" Verification

At the end of the session, the teacher's app fetches the **Exception List** from the Django backend.

- **Visual Audit:** The teacher's app displays the **Photos and Names** of only those few students on the exception list (e.g., 3 students out of 60).
- **Physical Identification:** The teacher asks those 3 students to stand up.
- **One-Tap Approval:** With a quick visual match, the teacher taps "Mark Present" on the dashboard. This takes **seconds**, rather than minutes, and keeps the attendance record 100% accurate.

Additional Features and Student-Faculty Utilities

Beyond the core attendance marking mechanism, the system includes several supporting features designed to make the entire attendance management experience seamless for both students and faculty.

Attendance Goal Tracker

Every student's dashboard displays a live calculation of exactly how many more lectures they need to attend to reach or maintain the 75% attendance threshold. This removes all guesswork — a student always knows their current standing and how much room they have before falling into the danger zone. The calculation updates automatically after every session.

Low Attendance Alerts for Faculty

The faculty dashboard flags students whose attendance is approaching or has fallen below the 75% threshold. The teacher can send a reminder directly to those students through the app with a single tap. This allows early intervention rather than discovering the problem only at the end of semester when it is too late for the student to recover.

Visual Analytics: The dashboard generates real-time graphs and attendance trends, helping teachers see engagement patterns to improve their lecture delivery.

Class Schedule Management

Faculty can update class timings, reschedule lectures, or announce cancellations directly through the app. All enrolled students receive an instant notification whenever the schedule changes. This eliminates the problem of students arriving for a class that was moved or missing a rescheduled session they were unaware of.

Digital Attendance Proposal for Valid Absences

If a student misses a lecture for a legitimate reason such as a medical issue, participation in an academic event, or any other valid cause, they can submit a digital attendance proposal directly through the app. The proposal includes the reason, supporting details, and any attachments such as a medical certificate or event participation letter. The faculty receives the proposal on their phone and can approve or reject it immediately. Since the proposal is submitted and reviewed digitally, there is no need for the student to physically visit the faculty or submit paper forms. The approved proposal is automatically reflected in the student's attendance record.

Group Proposal for Institutional Events

When a large group of students misses a lecture due to a common institutional activity — such as a college event, sports competition, technical fest, or inter-college trip — any one student can create a common group proposal for that event. Other students who were part of the same activity can attach themselves to that single proposal rather than submitting individual requests. The faculty reviews and approves the group proposal once, and attendance is updated for all attached students simultaneously. This makes the process significantly faster for events that involve many students at once.

Conclusion-

Our system transforms attendance from a 15-minute administrative burden into a seamless, **30-second digital ripple**. By replacing manual paper signing and expensive hardware with an automated **proximity web**, we eliminate proxy cheating while ensuring high reliability through a "peer-to-peer" architecture

This modern approach makes life easier for both students and faculty by providing a "zero-friction" experience that captures attendance in the background, allowing the lecture to start immediately. By saving thousands of hours of teaching time across the academic year, the project moves Indian universities away from outdated paperwork toward a truly smart, efficient, and data-driven campus.